

Paragliding Intelligence Platform

Volume 06 - AI & ML Architecture v2 Technical

Forecast Correction, Digital Twins, Risk Intelligence, AI Briefings, MLOps

Version: v2 Technical Replacement

Status: implementation-oriented architecture draft

Scope: AI/ML services, model contracts, training data, serving design, evaluation, monitoring, safety controls

Table of Contents

1. Purpose and Competitive Thesis
2. AI/ML System Overview
3. Model Inventory and Ownership
4. Canonical ML Data Architecture
5. Feature Store Design
6. Training Data and Labeling
7. Leakage Prevention and Temporal Integrity
8. Forecast Correction Models
9. Thermal and Cloudbase Prediction
10. Risk Intelligence Models
11. Digital Twin Engine
12. AI Briefing and LLM Grounding
13. XC Route Ranking and Recommendation
14. Pilot Personalization Engine
15. Computer Vision for Webcams
16. Model Training Pipelines
17. Model Registry and Release Governance
18. Serving Architecture
19. Monitoring, Drift, and Quality Assurance
20. Security, Privacy, and Safety
21. MVP Implementation Slice
22. Roadmap and Engineering Backlog
23. Appendices

1. Purpose and Competitive Thesis

This volume defines the AI and machine-learning architecture for a paragliding-first meteo application designed to outperform generic weather map applications and flight-computer apps by converting weather data into site-specific, pilot-specific decisions. The core differentiator is not prettier weather animation. The differentiator is a self-learning decision layer that answers whether a launch is worth driving to, what the best launch window is, what invisible risks exist, and how reliable the forecast is for that exact site.

The architecture deliberately separates deterministic safety rules, statistical machine learning, and natural-language AI. Deterministic blockers prevent unsafe recommendations. ML corrects forecasts and estimates site-specific variables. The LLM explains the final decision but is never allowed to invent weather values or override safety blockers.

Competitor baseline	Typical capability	Gap exploited by this platform
Windy / Ventusky style apps	Weather layers, animations, model selection, radar, satellite, broad visualization	Do not know each paragliding launch, pilot skill, local rotor traps, launch orientation, or historical forecast error for the exact site.
XCTrack / flight computer apps	In-flight navigation, vario integration, tasks, IGC logs, airspace, live flight utility	Usually not a full pre-flight weather intelligence and digital-twin forecasting platform.
Local paragliding weather pages	Site notes, webcams, simple local station values	Limited global scale, limited AI, limited ML correction, limited personal recommendations.
Target product	Weather + digital twins + ML correction + risk engine + AI briefing	Provides GO / MAYBE / NO-GO decisions with transparent reasons and confidence.

2. AI/ML System Overview

The AI/ML platform is organized as a layered decision system. Each layer has clear ownership and a strict data contract. This prevents the LLM from becoming a black-box weather oracle and keeps safety-critical decisions auditable.

Raw weather + terrain + observations + flights + pilot profile

|

v

Feature Store / Forecast Cube / Site Digital Twin

|

+--> Deterministic safety blockers

+--> Forecast correction ML

+--> Thermal/cloudbase ML

+--> Risk classifiers

+--> XC route ranking

+--> Personalization engine

|

v

Decision composer: GO / MAYBE / NO-GO + confidence + reasons

|

v

LLM briefing generator: explains only grounded facts and rules

Layer	Primary job	May block flight?	May use ML?	May use LLM?
-------	-------------	-------------------	-------------	--------------

Data ingestion	Collect forecasts, stations, radar, terrain, flights, observations	No	No	No
Feature store	Create point-in-time features for training and serving	No	No	No
Safety blockers	Apply hard rules: thunder, excessive gusts, wrong direction, no visibility	Yes	No by default	No
ML correction	Correct model bias for launch/site/time/terrain	No, but may reduce score	Yes	No
Risk intelligence	Estimate rotor, valley wind, foehn, overdevelopment, turbulence	Yes through risk thresholds	Yes	No
Decision composer	Combine blockers, scores, confidence, pilot profile	Yes	Uses ML outputs	No
AI briefing	Explain the decision and summarize data	No	No	Yes, grounded only

3. Model Inventory and Ownership

Every model must have a named owner service, a clear input feature schema, an output schema, a fallback path, and an evaluation metric. The platform will start with interpretable gradient-boosted models and deterministic formulas, then add deep-learning or graph-based methods only after the training corpus is large enough.

Model ID	Model	Owner service	Primary output	Fallback
ML-001	Launch wind correction	forecast-correction-service	Corrected wind speed/direction/gust at launch	Raw ensemble median + conservative penalty
ML-002	Cloudbase refinement	weather-ml-service	Corrected cloudbase AGL/MSL	LCL formula from temp/dewpoint + model cloudbase
ML-003	Thermal potential	thermal-engine-service	Thermal strength, trigger probability	Deterministic radiation/slope heuristic
ML-004	Rotor/turbulence risk	risk-engine-service	Rotor risk 0-100, reason codes	Terrain/wind-direction rules
ML-005	Valley wind risk	risk-engine-service	Valley wind amplification estimate	Local rule table
ML-006	Foehn/mountain wave classifier	risk-engine-service	Foehn/wave probability	Pressure gradient thresholds
ML-007	Overdevelopment/storm growth	nowcast-risk-service	OD risk and time-to-danger	CAPE/radar/lightning thresholds
ML-008	XC potential ranker	xc-planner-service	Route score and expected distance	Rule-based route scoring
ML-009	Pilot personalization	recommendation-service	Personalized GO/MAYBE/NO-GO adjustment	Skill-tier defaults
ML-010	Webcam cloud/windsock CV	vision-service	Visibility/cloud/windsock signal	Manual webcam display only
AI-001	Grounded flight briefing	ai-briefing-service	Human-readable briefing	Template renderer with no LLM
AI-002	AI debrief coach	ai-coach-service	Post-flight debrief	Structured IGC metrics report

4. Canonical ML Data Architecture

The ML platform uses point-in-time training data. Every feature must carry the forecast run time, valid time, ingestion time, source model, and whether the value was available before the prediction. This is critical: paragliding decisions are time-sensitive and the system must never train on future observations or post-flight outcomes as if they were pre-flight inputs.

entity keys:

site_id: launch or landing identifier

forecast_run_id: source model run, e.g. GFS 2026-06-04 00Z

valid_time: forecast target timestamp

feature_generated_at: platform processing timestamp

observation_time: timestamp for station/pilot/webcam observation

pilot_profile_id: optional personalization entity

time integrity rule:

training_row.prediction_time <= all input_feature.available_at

labels may occur after prediction_time, but must never be joined as features

Data object	Storage	Purpose
forecast_model_runs	PostgreSQL	Tracks model source, run cycle, horizon, processing status, quality flags
site_forecasts	TimescaleDB hypertable	Canonical hourly site forecasts and corrected values
grid_forecast_tiles	Object storage + tile cache	Map layers and forecast cubes for WebGL animation
station_observations	TimescaleDB hypertable	Ground-truth wind, gust, temperature, pressure, humidity
pilot_observations	PostgreSQL/Timescale	Crowd-sourced reality reports and quality/confidence
igc_flights	Object storage + PostGIS metadata	Training labels for XC quality, climb rates, route choices
feature_snapshots	Object storage/Parquet	Immutable point-in-time training datasets
model_predictions	PostgreSQL/Timescale	Auditable predictions and later realized outcomes

5. Feature Store Design

The project can start with SQL/Parquet feature views and later adopt a dedicated feature store such as Feast when online/offline consistency becomes hard to manage manually. The feature store is not a magic ML layer; it is a governance mechanism that ensures the same feature definitions are used during training and production inference.

Feature group	Example features	Online?	Offline?	Owner
site_static	launch altitude, aspect, safe wind sectors, hazard tags, ridge geometry	Yes	Yes	site-service
forecast_dynamic	wind by altitude, gust, temperature, dewpoint, precipitation, CAPE	Yes	Yes	forecast-processing-service
terrain_context	slope/aspect sectors, valley axis, ridge exposure, DEM roughness	Yes	Yes	terrain-service
reality_dynamic	station delta, pilot reports, webcam visibility, radar proximity	Yes	Yes	reality-service
history_features	past forecast bias, similar-day outcomes, site error distribution	Yes	Yes	digital-twin-service
pilot_profile	skill tier, wing class, max accepted wind/gust, preferred XC distance	Yes	Restricted	profile-service

feature view: site_forecast_context_v1

entities: [site_id, valid_time]

features:

wind_surface_kmh, wind_500m_kmh, wind_1000m_kmh

gust_surface_kmh, gust_spread_kmh

wind_dir_surface_deg, wind_dir_1000m_deg

cloudbase_m_agl, precipitation_mm_h, cape_j_kg

model_disagreement_wind_kmh

site_forecast_bias_wind_kmh_30d

station_delta_wind_kmh_2h

terrain_exposure_index

valley_alignment_score

6. Training Data and Labeling

Labels are generated from observations, flights, reports, and incident-free outcomes. The platform must distinguish between weather labels and human-decision labels. A site being flown successfully does not automatically mean the conditions were safe for beginners. Labels therefore include objective measurements and tier-specific outcomes.

Label	Definition	Positive source	Negative source	Notes
actual_launch_wind_kmh	Observed wind at/near launch valid time	Launch station, trusted pilot report	N/A	Used for wind correction regression
actual_gust_kmh	Maximum gust during decision window	Station gusts	N/A	Used for gust spread correction
thermal_strength_ms	Median climb rate from IGC tracks after filtering engine/tow data	IGC climb segments	Weak/no climb windows	Needs altitude smoothing
flyable_intermediate	Conditions acceptable for intermediate pilots	Validated flights, instructor reports	Warnings, no-fly reports, incidents	Classification label
rotor_reported	Rotor/turbulence reported in site sector	Pilot report, incident report	No turbulence reports with sufficient flights	Sparse label; start with rules
xc_success_km	Distance potential under conditions	IGC routes	No meaningful XC flights despite attempts	Affected by pilot skill; normalize
briefing_quality	Human rating of AI briefing usefulness	Post-briefing feedback	Low rating or correction	Used for prompt templates, not weather truth

7. Leakage Prevention and Temporal Integrity

Weather and trading-style systems both suffer from leakage when future outcomes are included in training features. This architecture enforces temporal joins and leakage audits. Every training dataset must be reproducible from a prediction timestamp and only use values available at or before that timestamp.

anti-leakage rules:

1. Do not join station observations after prediction_time as features.
2. Do not join IGC flight outcomes as features for the same decision window.
3. Do not use revised forecast products unless revision availability is recorded.
4. Do not train and test on randomly split rows across the same site/time season.
5. Use walk-forward validation by date and holdout sites by geography.
6. Store feature_snapshot_id for every model training run.
7. Keep label_generation_sql under version control.

Validation split	Purpose	Expected use
Walk-forward time split	Tests future performance after training on past seasons	Default for model promotion
Site holdout split	Tests new-site generalization	Required before global expansion
Mountain-region split	Tests terrain domain shift	Required for Alps/Norway/Pyrenees/Carpathians
Storm-season split	Tests rare-risk sensitivity	Required for safety classifiers
Cold-start split	Tests sites with no digital twin history	Required for MVP launch database expansion

8. Forecast Correction Models

Forecast correction is the first high-value ML feature. The product should not claim to replace professional numerical weather prediction. It should learn local bias: model underestimates valley wind, overestimates launch wind behind ridges, or misses gust amplification for specific sectors.

Target	Model type MVP	Model type V2	Key features	Metric
wind_speed_launch_kmh	XGBoost/LightGBM regression	Quantile GBDT + site embeddings	raw forecast, altitude, terrain, station deltas, season, hour	MAE, P90 absolute error
wind_direction_launch_deg	Circular regression or sin/cos regression	Mixture model by sector	u/v components, topography, valley axis	Mean angular error
gust_launch_kmh	Quantile regression	Ensemble quantile model	raw gust, wind shear, stability, exposure	Pinball loss, P90 coverage
cloudbase_m_agl	GBDT regression	Hybrid LCL + ML residual	temp, dewpoint, humidity, cloud cover, terrain	MAE, conservative bias
model_confidence	Calibrated error model	Conformal intervals	model disagreement, past site bias, observation density	Calibration error

```
def corrected_wind(site_id, valid_time, forecast_features, reality_features):

    base = forecast_features['wind_surface_kmh']

    residual = wind_correction_model.predict_residual(forecast_features)

    p90_error = error_model.predict_p90_abs_error(forecast_features)

    corrected = max(0, base + residual)

    return {

        'wind_kmh': corrected,

        'uncertainty_p90_kmh': p90_error,

        'model_version': 'wind-correction-xgb-v1.3.0'

    }
```

9. Thermal and Cloudbase Prediction

Thermal prediction is a major differentiator because paragliding pilots care about climb usability, not only wind. The thermal engine combines deterministic physics proxies with ML residual correction. Early versions should be transparent and conservative because thermal labels from IGC files are noisy.

Component	Feature examples	Computation
Solar forcing	global radiation, sun angle, terrain aspect, slope	DEM + solar position + cloud cover
Surface trigger potential	land cover, rocky/snow/forest/urban class, ridge shape	GIS raster features
Atmospheric usability	lapse rate proxy, CAPE, boundary layer height, inversion flags	Forecast vertical profile if available
Cloud suppression	low/mid/high cloud cover, humidity, precipitation risk	Forecast fields
Label from IGC	smoothed climb rate, climb duration, altitude band	Track parser + noise filtering
Output	thermal_strength_ms, trigger_probability, thermal_confidence	ML + conservative clipping

```
thermal_potential = sigmoid(

    0.35 * normalized_solar_radiation

    + 0.20 * slope_aspect_alignment

    + 0.15 * landcover_trigger_score
```

```
+ 0.15 * boundary_layer_depth_score

- 0.25 * cloud_suppression

- 0.30 * precipitation_penalty

)
```

10. Risk Intelligence Models

Risk models must be designed as safety assistance, not as permission to fly. A site may have high flyability but still have a high risk score. The decision composer must expose this contradiction instead of hiding it inside a single score.

Risk	Detection MVP	ML evolution	Hard blocker examples
Rotor	Wind direction vs ridge/launch geometry + wind speed threshold	Classifier trained on reports/incidents and turbulent IGC signals	Rotor risk > 85 for pilot tier
Foehn	Pressure gradient across mountain range + dry/warm downslope wind	Regional classifier with station gradients	Foehn flag high and gusts increasing
Valley wind	Valley-axis alignment + time-of-day + temperature gradient	Site-specific amplification model	Predicted valley wind exceeds landing limit
Overdevelopment	CAPE, CIN, humidity, radar growth, lightning proximity	Nowcast classifier	Lightning within configurable radius
Mountain wave	Wind speed at ridge altitude + stability + cross-ridge flow	Wave probability model	Wave/high turbulence warning in mountain sites
Low cloud/visibility	Cloudbase below launch/terrain clearance + webcam visibility	CV + forecast fusion	Cloudbase below safe terrain clearance

```
risk_score = max(

    rotor_risk,

    foehn_risk,

    storm_risk,

    low_cloud_risk,

    valley_wind_risk,

    mountain_wave_risk

)

if storm_risk >= 90 or lightning_distance_km <= 25:

    decision = 'NO_GO'

elif risk_score >= pilot_profile.max_accepted_risk:

    decision = 'NO_GO'

else:

    decision = weighted_score_decision(...)
```

11. Digital Twin Engine

A launch-site digital twin is the accumulated local intelligence for a specific launch and landing system. It is not a 3D simulation in the MVP. It is a continuously updated statistical memory of forecast errors, safe/unsafe sectors, thermal outcomes, pilot reports, station deltas, and known local hazards.

Digital twin state	Updated from	Used by
forecast_bias_by_sector	station observations, pilot reports	forecast correction
gust_amplification_by_hour	stations, reports	risk engine
thermal_trigger_history	IGC climbs, reports	thermal engine
safe_sector_confidence	verified site metadata, observations	flyability engine
hazard_evidence	reports, admin review, incidents	briefing and risk engine
model_skill_by_season	validation jobs	confidence scoring
cold_start_similarity	terrain/site embeddings	new-site bootstrap

site_digital_twin_update event:

```
{
  "site_id": "uuid",
  "valid_time": "2026-06-04T13:00:00Z",
  "signal_type": "forecast_error",
  "source": "station_observation",
  "features": {"forecast_wind_kmh": 15.0, "observed_wind_kmh": 22.0},
  "delta": {"wind_error_kmh": 7.0},
  "quality_weight": 0.92
}
```

12. AI Briefing and LLM Grounding

The AI flight briefing is a user-experience layer over deterministic and ML outputs. It must never be the source of weather truth. The LLM receives a compact, typed context object and must produce a briefing constrained to those values. The prompt explicitly forbids inventing weather, changing the decision, or minimizing risk.

Guardrail	Implementation
No invented data	LLM receives only JSON context; response must reference context values.
No override	Decision composer sets GO/MAYBE/NO_GO before LLM call. LLM cannot change it.
Risk-first language	Prompt instructs the model to mention blockers and highest risks before positive conditions.
Citation to data cards	Briefing sections map to forecast, risk, site, and pilot profile data cards.
Template fallback	If LLM fails validation, render deterministic template.
JSON schema validation	LLM output must match briefing schema before display.
Paid-tier gating	Advanced AI briefing, XC coach, and debrief require entitlement checks.

AI briefing input context:

```
{
  "decision": "MAYBE",
  "site": {"name": "Vaga Launch", "safe_directions": ["NW", "N", "NE"]},
  "pilot_profile": {"skill": "intermediate", "wing_class": "EN-B"},
  "best_window": {"start": "12:00", "end": "15:00"},
  "forecast": {"wind_kmh": 18, "gust_kmh": 27, "direction": "NW", "cloudbase_m_agl": 1300},
  "risks": [{"type": "gust_spread", "score": 62, "reason": "gust spread increases after 15:00"}],
  "hard_blockers": []
}
```

}

13. XC Route Ranking and Recommendation

XC route recommendation is not autonomous flight instruction. It ranks plausible corridors by weather usability, terrain, airspace, landing options, and pilot skill. In MVP the engine should generate ranked route suggestions from known waypoints and terrain corridors rather than free-form routes.

Input	Description
launch_site_id	Start site selected by user or recommendation engine
pilot_skill_tier	Beginner/intermediate/XC/competition constraints
weather_corridor_cube	Wind, cloudbase, thermal potential, storm risk along route grid
terrain/landing_data	Terrain clearance and emergency landing density
airspace_constraints	Controlled/restricted zones and altitude ceilings
target_distance	20 km, 50 km, 100 km, competition task, out-and-return

```
route_score = (  
  
    0.25 * thermal_chain_score  
  
+ 0.20 * tailwind_or_crosswind_compatibility  
  
+ 0.15 * cloudbase_clearance_score  
  
+ 0.15 * landing_option_score  
  
+ 0.10 * airspace_clearance_score  
  
+ 0.10 * model_confidence  
  
- 0.25 * max_risk_along_route  
  
)
```

14. Pilot Personalization Engine

Personalization adjusts the interpretation of conditions, not the underlying weather. A beginner and a competition pilot may see different recommendations for the same site, but the raw values and risk reasons must remain visible. The platform should default to conservative profiles until the user explicitly configures experience and wing class.

Profile field	Use in decision	Default
skill_tier	Adjusts wind/gust/risk thresholds	beginner if unknown
wing_class	Adjusts speed/risk tolerances and warning language	unknown -> conservative
recent_flight_count	Limits confidence in advanced recommendations	0
max_wind_kmh	User-specific alert threshold, bounded by site safety limits	site beginner default
max_gust_kmh	User-specific gust threshold	site beginner default
xc_goal_km	Route ranking target	local soaring
risk_tolerance	Only narrows display recommendations; cannot disable hard blockers	normal/conservative

15. Computer Vision for Webcams

Webcam AI is a V2/V3 feature. It can provide high-value live reality signals: visibility, cloud cover, low cloud on launch, precipitation on lens, and windsock orientation when visible. The first implementation should not make autonomous safety decisions from a webcam alone. It should add confidence or warnings to the live reality layer.

CV task	Model approach	Output	Safety use
Cloud/visibility classification	Lightweight CNN or vision	visibility class, cloud obstruction	Adds low-visibility warning

	transformer fine-tuned on site frames		
Windsock detection	Object detection + angle estimation	approx wind direction/sock extension	Secondary signal only
Rain/snow detection	Image classifier + radar confirmation	precipitation likelihood	Nowcast warning
Camera obstruction	Classifier for dark/frozen/covered lens	camera health	Do not use camera signal
Cloud growth trend	Frame differencing + classifier	cloud build-up trend	Overdevelopment signal

16. Model Training Pipelines

Training jobs are batch-oriented and reproducible. Each job takes a feature snapshot, label definition version, code version, and hyperparameter configuration. The output is a model artifact plus model card, metrics, calibration report, and promotion recommendation.

training pipeline stages:

1. select training window and sites
2. materialize point-in-time feature snapshot
3. generate labels using versioned SQL/Python
4. run data-quality and leakage checks
5. train baseline and candidate models
6. evaluate walk-forward and site-holdout splits
7. calibrate confidence intervals / probabilities
8. create model card and registry entry
9. deploy to shadow mode
10. promote only after live validation

Artifact	Format	Stored in
feature snapshot	Parquet + manifest JSON	object storage
model artifact	native model + optional ONNX	model registry
model card	Markdown/JSON	model registry + docs
evaluation report	HTML/PDF/JSON	model registry
calibration curves	PNG/JSON	object storage
shadow predictions	TimescaleDB/PostgreSQL	model_predictions table

17. Model Registry and Release Governance

The registry controls lifecycle state: development, staging, shadow, production, deprecated, and blocked. A model may only serve production traffic when it passes offline tests, shadow-mode tests, and safety review. The model card must explain intended use, limitations, training data, validation splits, and known failure cases.

Promotion gate	Required evidence
Offline performance	Improves relevant baseline and does not worsen safety-critical error metrics
Calibration	Confidence intervals or probabilities are calibrated within tolerance
Cold-start behavior	Fallback path behaves conservatively for low-data sites
Shadow validation	Runs on live data without user-visible effect for configured period
Safety review	No known risk-reducing bias in dangerous weather categories
Rollback plan	Previous model version remains available and tested

18. Serving Architecture

Serving must be low-latency for site pages and alerts. Heavy model calculations can run during forecast processing and store outputs. Real-time serving is reserved for personalization, alerts, AI briefing, and user-specific route queries.

forecast run completed

|

v

batch inference: correction + thermal + risk + confidence

|

v

site_forecasts + risk_assessments + feature snapshots

|

+--> alert evaluator

+--> map tile generator

+--> site forecast API

+--> AI briefing context builder

Serving mode	Use case	Latency target	Technology
Batch inference	All site forecasts after new model run	minutes after forecast ingestion	Celery/Kubernetes jobs
Online inference	User-specific flyability, personalization, route request	<500 ms internal	FastAPI model service
Streaming inference	Live tracking alerts, storm/airspace warnings	<5 s for tracking warnings	WebSocket + stream processor
LLM generation	AI briefing/debrief	2-10 s acceptable	LLM provider or private LLM with schema validation
Fallback templates	Briefing when LLM unavailable	<200 ms	Template renderer

19. Monitoring, Drift, and Quality Assurance

The ML platform requires two monitoring layers: operational monitoring and model-quality monitoring. Operational metrics detect service failure. Model-quality metrics detect forecast degradation, station outages, drift, and unsafe model behavior.

Metric	Alert condition	Owner
feature_freshness_minutes	Feature group older than threshold	feature-platform
forecast_correction_mae_24h	Rolling MAE exceeds seasonal baseline	weather-ml
gust_underprediction_rate	Underprediction above threshold in high-wind windows	risk-engine
risk_blocker_false_negative_review	Any credible report of dangerous condition not warned	safety review
llm_schema_failure_rate	Schema validation failure > threshold	ai-briefing
briefing_disagreement_count	LLM text contradicts structured context	ai-briefing
model_latency_p95	p95 exceeds SLO	platform
data_gap_station_count	Critical station offline	reality-service

20. Security, Privacy, and Safety

AI/ML features handle sensitive location and behavior data, especially live tracking, pilot profiles, and flight logs. Model training must use privacy-aware aggregation. Personalization should not expose private flight history to clubs, schools, or public pages unless explicitly shared.

- Store live tracking with explicit consent and configurable retention.
- Separate personal pilot features from public site features.
- Do not train public models on private IGC tracks unless the user opts in or data is anonymized according to policy.
- Log every AI decision context object for audit, but avoid storing full private conversations unless necessary.
- Safety language must be conservative and include uncertainty. The product provides decision support, not official aviation clearance.
- Admin tools must allow disabling a model, a launch recommendation, or an entire weather source when quality is compromised.

21. MVP Implementation Slice

The MVP should avoid overbuilding deep ML before data exists. It should ship with strong data architecture, deterministic safety scoring, simple correction models where observations exist, and AI briefing grounded in structured data.

MVP item	Implementation detail	Reason
Feature snapshots	Materialize site-hour features to Parquet	Prevents future leakage and enables later ML
Rule-based flyability	Deterministic GO/MAYBE/NO-GO with site limits	Trustworthy before sufficient labels
Simple forecast bias correction	Rolling site/station bias where observations exist	Immediate value with low risk
Thermal heuristic	Solar/slope/cloud heuristic	Useful before IGC corpus grows
Risk blockers	Thunder, gust, wrong direction, low cloud, rain	Safety-first MVP
Grounded AI briefing	Template + optional LLM with schema validation	User delight without hallucination risk
Shadow prediction table	Store ML predictions and later outcomes	Build learning loop from day one

22. Roadmap and Engineering Backlog

Phase	AI/ML capability	Exit criteria
MVP	Rule scoring, bias correction, grounded AI briefing, feature snapshots	20-50 pilot sites, reliable forecast pages, auditable decisions
V1	Launch wind correction, cloudbase residual model, validation dashboard	MAE improvement vs raw model on observed sites
V2	Thermal model trained on IGC, digital twin profiles, site-holdout evaluation	XC score improves route/site ranking
V3	Webcam CV, nowcast storm growth, route ranking, AI debrief	Live reality layer influences decisions safely
V4	Global site similarity model, rescue drift modeling, competition optimization	Enterprise/club/school workflows supported

23. Appendices

Appendix A - Example Model Card

model_name: launch-wind-correction-xgb

version: 1.3.0

owner: forecast-correction-service

intended_use: Correct surface launch wind forecast for known paragliding sites.

not_intended_use: Official aviation forecast, autonomous launch approval, storm warnings.

training_data: site-hour feature snapshots from 2024-2026, station observations, trusted pilot reports.

validation: walk-forward, site-holdout, region-holdout.

metrics: MAE, P90 absolute error, high-wind underprediction rate.

fallback: raw ensemble median + conservative uncertainty penalty.

known_failures: cold-start sites without stations, rapidly developing convective storms, station obstruction.

Appendix B - Example SQL Tables

```
CREATE TABLE ml_model_versions (  
  
    id UUID PRIMARY KEY,  
  
    model_name TEXT NOT NULL,  
  
    version TEXT NOT NULL,  
  
    owner_service TEXT NOT NULL,  
  
    lifecycle_stage TEXT NOT NULL CHECK (lifecycle_stage IN ('dev','staging','shadow','production','deprecated','blocked')),  
  
    artifact_uri TEXT NOT NULL,  
  
    feature_snapshot_uri TEXT,  
  
    metrics JSONB NOT NULL DEFAULT '{}',  
  
    model_card JSONB NOT NULL DEFAULT '{}',  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
  
);
```

```
CREATE TABLE ml_predictions (  
  
    id UUID PRIMARY KEY,  
  
    model_version_id UUID REFERENCES ml_model_versions(id),  
  
    site_id UUID NOT NULL,  
  
    prediction_time TIMESTAMPTZ NOT NULL,  
  
    valid_time TIMESTAMPTZ NOT NULL,  
  
    prediction_type TEXT NOT NULL,  
  
    input_context JSONB NOT NULL,  
  
    output JSONB NOT NULL,  
  
    observed_outcome JSONB,  
  
    outcome_available_at TIMESTAMPTZ  
  
);
```

```
CREATE INDEX idx_ml_predictions_site_time ON ml_predictions (site_id, valid_time DESC);
```

```
CREATE INDEX idx_ml_predictions_type_time ON ml_predictions (prediction_type, valid_time DESC);
```

Appendix C - Example LLM Output Schema

```
{  
  
    "type": "object",  
  
    "required": ["decision", "summary", "best_window", "top_risks", "do_not_fly_if", "confidence_note"],
```

```
"properties": {  
  
  "decision": {"enum": ["GO", "MAYBE", "NO_GO"]},  
  
  "summary": {"type": "string", "maxLength": 700},  
  
  "best_window": {"type": "string"},  
  
  "top_risks": {"type": "array", "items": {"type": "string"}},  
  
  "do_not_fly_if": {"type": "array", "items": {"type": "string"}},  
  
  "confidence_note": {"type": "string", "maxLength": 300}  
  
}
```

Appendix D - Reference Technologies

- MLflow Model Registry - model lifecycle, versioning, aliases, tags, metadata and annotations: <https://mlflow.org/docs/latest/ml/model-registry/>
- Feast Feature Store - define, manage and serve ML features: <https://docs.feast.dev/>
- XGBoost documentation - gradient boosting library and Python API: <https://xgboost.readthedocs.io/>
- scikit-learn model persistence - ONNX and persistence tradeoffs: https://scikit-learn.org/stable/model_persistence.html
- Open-Meteo forecast API - multi-model weather forecast data: <https://open-meteo.com/en/docs>
- MET Norway APIs - locationforecast and public weather data: <https://api.met.no/>
- ECMWF Open Data - global forecast data for reuse with attribution: <https://www.ecmwf.int/en/forecasts/datasets/open-data>
- NOAA/NCEP NOMADS - GFS and operational model data access: <https://nomads.ncep.noaa.gov/>
- OpenAIP API - aeronautical data access for airspace: <https://www.openaip.net/>